Opticka Documentation

Ian Max Andolina

2026-06-18

Contents

Frontmatter

Opticka is an object-oriented framework with optional GUI for the Psychophysics toolbox (PTB), allowing full experimental presentation of complex visual or other stimuli. It is designed to work on Linux, macOS or Windows. It interfaces via strobed words and/or ethernet for recording neurophysiological and behavioural data. Full behavioural task control is available by use of a Finite State-Machine controller, in addition to simple method of constants (MOC) experiments.

DOI 10.5281/zenodo.592253 release V2.17.0 commits since V2.17.0 88

last commit yesterday DeepWiki Docs Open in Visual Studio Code Maintained? yes

License GPLv3

Opticka's Task Modes

Opticka's Two Experiment Modes

1. **Behavioural Task** – a behavioural task uses a `stateMachine` to construct a series of experiment states and the transitions between them. It uses `StateInfo.m` files and `userFunctions.m` files to specify the states and any customised functions required. It can optionally use `taskSequence`, `metaStimulus`, `eyeTracker`, `IO`, and other manager classes to control stimuli, variables, and hardware interaction.
2. **Method of Constants (MOC) Task** – a MOC task is a task where **no** behavioural control is required. It uses `metaStimulus` and `taskSequence` to define a set of stimuli and the variables that will drive unique trials repeated over blocks.

Comparison

Feature	Behavioural Task	MOC Task
State machine	Yes (`stateMachine` + `StateInfo.m`)	No
Eyetracker/touch	Supported	Not used
Trial flow	Transition-driven (fixation, response)	Timer-driven
Feedback states	Correct / incorrect / breakfix	None
Data saving	Per-trial with response codes	Per-trial stimulus data only

Basic Settings (see below for more details):

- **Repeat Blocks**: number of blocks to repeat.
- **Random Algorithm**: the randomisation algorithm used by MATLAB's rand functions.
- **MOC Trial time**: the time the stimulus is presented for.
- **MOC IT time**: the time between trials.
- **MOC IB time**: the time between blocks.
- **REAL Time**: We can use two timing methods, when REAL Time=ON then we use the PTB method `GetSecs` or the VBL to measure time stamps and task time is based on these real time values. When REAL Time=OFF then we calculate how many frames a time interval should contain and use ticks to measure time as each frame is shown. REAL Time is essential for real experiments, ticks are more useful for debugging where you can stop and step through carefully.

Task Parameters

Opticka uses the `taskSequence` class to build the stimulus randomisation (see here). This class takes a series of **independent variables** with values and builds the randomisation into repeated blocks. The GUI does this using the Ind. Variable List editor, but the underlying code looks like this (`nBlocks` is **Task Options > Repeat Blocks** in the GUI):

```
% task sequence with 2 blocks
task = taskSequence('nBlocks', 2);

% first variable: 3 angles that will be applied to stimuli 1 & 2
```

```
task.nVar(1).name = 'angle';
task.nVar(1).values = [-25 0 25];
task.nVar(1).stimulus = [1 2];

% second variable: 2 colours that will be applied to stimulus 3 only
task.nVar(2).name = 'colour';
task.nVar(2).values = {[1 0 0], [0 1 0]};
task.nVar(2).stimulus = 3;

randomiseTask(task);
```

Independent variables can be any stimulus parameter (`size`, `angle`, `xPosition` etc.) that can be assigned to that stimulus. There are some magic variables like `xyPosition` that allows you to pass both the X and Y position in a single variable value.

Opticka's State Machine

Controlling Behavioural Tasks using State Machines

Any experiment can be thought of as a series of states (delay period, initiate fixation, present stimulus, give subject feedback, post-trial timeout, etc.) and conditions for switching between these states. State Machines are a widely used control system for these types of scenarios. The important definition of a state machine is:

1. The state machine must be in exactly one of a finite number of states at any given time. States run for a defined amount of time by default before switching to a `next` state.
2. The state machine can also change from one state to another based on rules; the conditional change from one state to another is called a **transition**. A function controls the transition (`transitionFcn`), returning a different state name that forces a transition to this new state.
3. Functions can run when we **enter** (`enterFcn`), are **within** (`withinFcn`), or **exit** (`exitFcn`) a state.

Opticka defines all of this in a `StateInfo.m` file that specifies function arrays and assigns them to a state list. The Opticka GUI visualises the state table and lists out each function array.

For example this cell array identifies two functions which draw and animate our `stims` stimulus object. The `@()` identifies these as MATLAB function handles:

```
{ @()draw(stims); @()animate(stims); }
```

Opticka has many core functions that can run during state machine traversal. You can additionally write your own functions and store them in a userFunctions.m class file.

As an example, the `DefaultStateInfo.m` file defines several experiment states (*prefix*, *fixate*, *stimulus*, *incorrect*, *breakfix*, *correct*, *timeout*) and how the task switches between them (either with a timer or transitioned using an eyetracker):

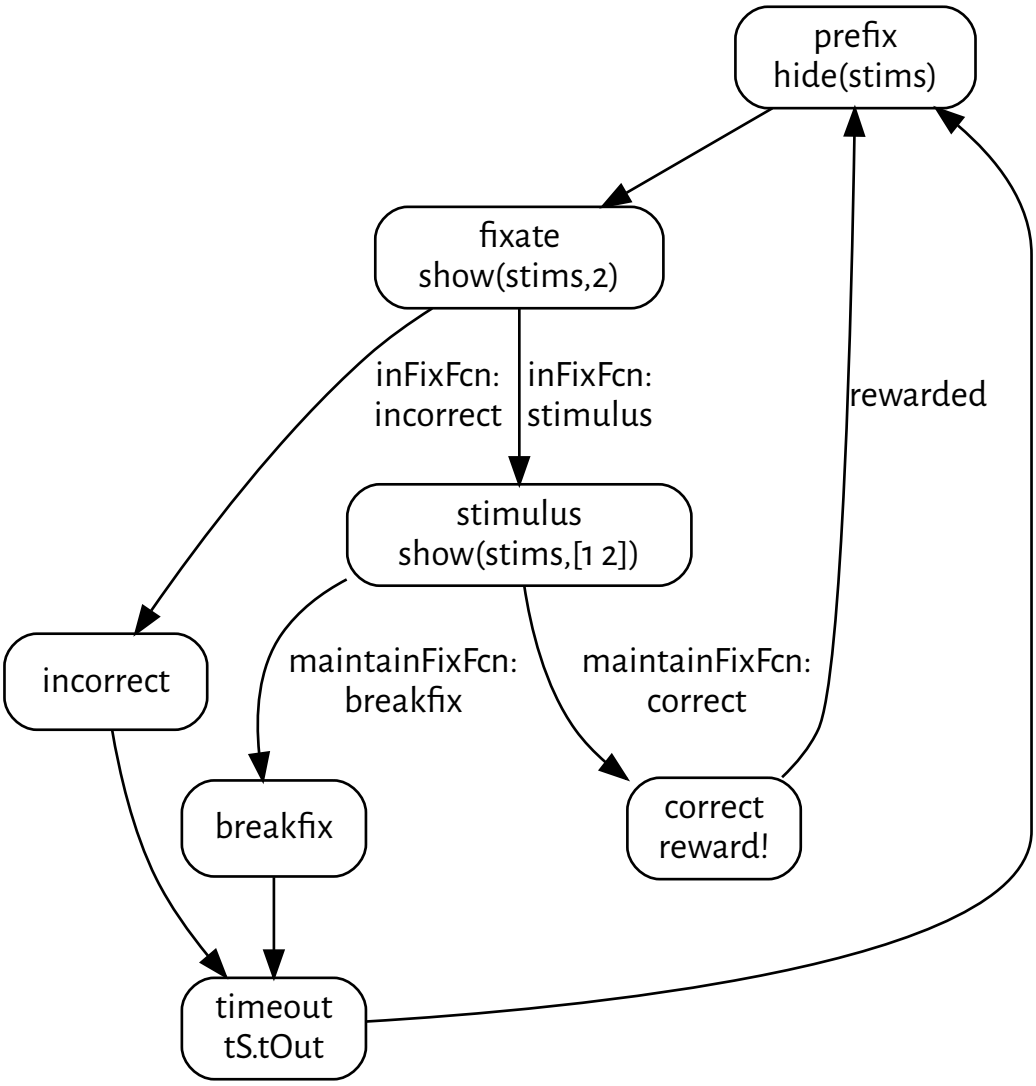
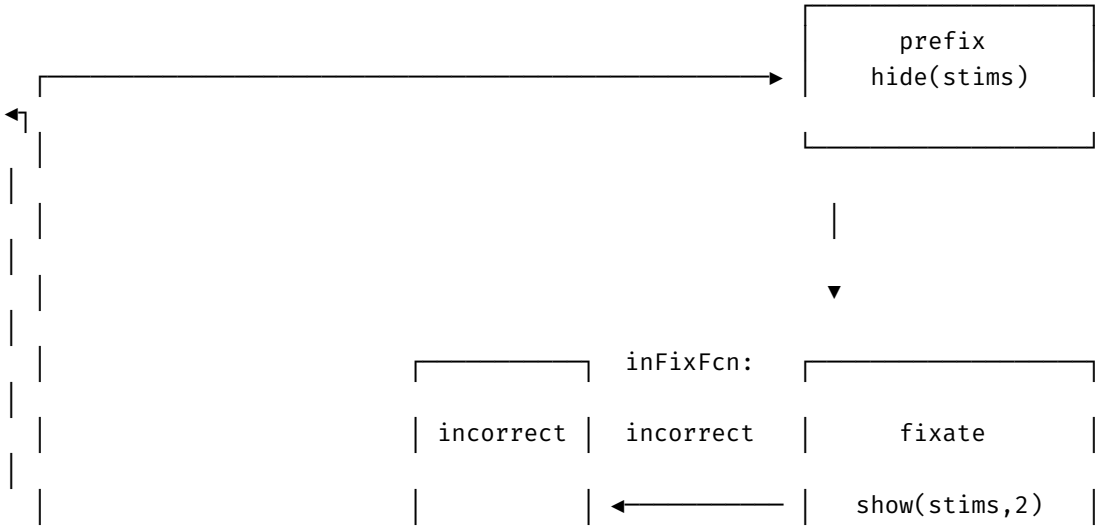
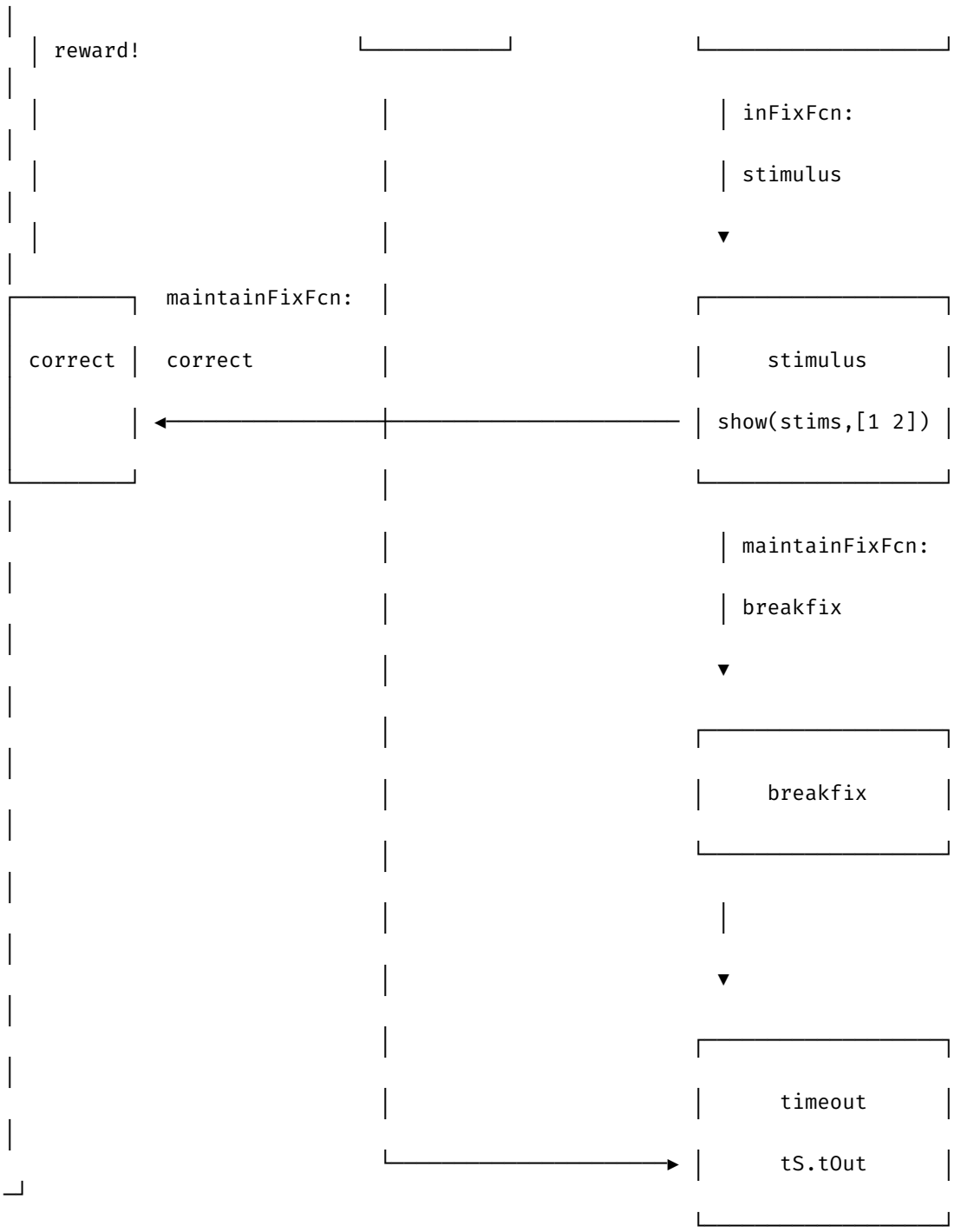


Figure 1: "An example state machine task."

The same state flow shown as an ASCII diagram:





State info files, being plain `.m` files, should be edited in the MATLAB editor (the GUI has an edit button that opens the file in the editor for you).

State Table Format

The state machine is defined as a cell array with 8 columns:

```
stateInfoTmp = {
'name'  'next'  'time'  'entryFcn'  'withinFcn'  'transitionFcn'  'exitFcn'
'HED';
```

```
... rows for each state ...
};
```

Column	Description
`name`	State name (string). Must be unique.
`next`	Default next state after the time expires.
`time`	Maximum time in seconds before auto-transition to `next`. Use `Inf` for no timeout.
`entryFcn`	Cell array of `@()` function handles run **once** on state entry.
`withinFcn`	Cell array of `@()` function handles run **every frame** while in this state.
`transitionFcn`	Cell array of `@()` function handles that return a state name string to force an early transition. Returns `''` to stay in current state.
`exitFcn`	Cell array of `@()` function handles run **once** on state exit.
`HED`	Hierarchical event descriptor tag for this state.

Example: fixate state definition

```
fixEntryFcn = { @()show(stims, 2); @()draw(stims) };
fixFcn      = { @()draw(stims); @()drawPhotoDiodeSquare(s, [0 0 0]) };
inFixFcn    = { @()testSearchHoldFixation(eT, 'stimulus', 'breakfix') };
fixExitFcn  = { @()updateFixationValues(eT, [], [], [], tS.stimulusFixTime);
                @()show(stims); @()trackerMessage(eT, 'END_FIX') };
```

Standard States

Most Opticka protocols use a common set of standard states. You can add, remove, or rename states as needed for your paradigm.

State	Purpose	Typical Time
`pause`	Task paused, screen blanked, ET stopped	`Inf` (manual resume)
`prefix` / `blank` / `prestim`	Pre-trial setup, ET initialisation, hide stimuli	0.2–0.5 s
`fixate` / `fixation`	Subject initiates and holds fixation	`Inf` (transition-driven)
`stimulus` / `sample`	Stimulus presentation while maintaining fixation	`Inf` (transition-driven)
`correct`	Reward + positive feedback	0.3–1.0 s
`incorrect`	Error feedback	0.3–0.5 s
`breakfix`	Fixation break feedback	0.3–0.5 s
`timeout`	Delay after error	0.5–2.0 s
`calibrate`	Eyetracker calibration	`Inf` (manual exit)
`drift`	Drift correction	`Inf` (manual exit)
`offset`	Drift offset	`Inf` (manual exit)
`override`	Debug override mode (keyboard control)	`Inf` (manual exit)
`flash`	Full-screen flash (photodiode calibration)	0.1 s
`showgrid`	1-degree grid display	`Inf` (manual exit)

Protocol-specific states

Some protocols add custom states:

State	Protocol	Purpose
`catchtrial`	SaccadePhosphene	Catch trial with no visual target
`exclusion`	SaccadePhosphene	Entered an exclusion zone
`delay`	DMTS	Blank delay period between sample and choice
`choice`	DMTS	Choice array with target + distractors
`magstim`	DotDirection, DotColour	Magnetic stimulation trigger

The `tS` Structure

The `tS` structure is created in the state info file and holds all the settings and constants used throughout the experiment. Common fields include:

Field	Description
`tS.name`	Protocol name
`tS.saveData`	Whether to save data after each trial
`tS.useTask`	Whether to use the task sequence
`tS.includeErrors`	Whether to advance the task on errors (vs. `resetRun`)
`tS.enableTrainingKeys`	Enable arrow-key stimulus adjustment
`tS.keyExclusionPattern`	Key names to exclude from task control
`tS.CORRECT` / `tS.INCORRECT` / `tS.BREAKFIX`	Numeric codes for trial results
`tS.correctSound` / `tS.errorSound`	Sound frequency for feedback beeps
`tS.fixX` / `tS.fixY`	Fixation window centre position (degrees)
`tS.firstFixInit`	Time allowed to search for fixation window
`tS.firstFixTime`	Required fixation hold time
`tS.firstFixRadius`	Fixation window radius
`tS.stimulusFixTime`	Fixation hold time during stimulus
`tS.strict`	Strict fixation mode (cannot leave window)
`tS.tOut`	Timeout duration after errors

Conditional Function Assembly

State info files often conditionally assemble function arrays based on `tS` settings. This allows a single state file to support multiple experimental modes.

includeErrors pattern

When `tS.includeErrors` is `true`, errors advance the task sequence. When `false`, `resetRun(task)` is called instead, randomising within the current block:

```
exitFcn = { @()updatePlot(bR,me); @()updateVariables(me); @()update(stims);
            @()resetAll(eT); @()plot(bR,1) };

if tS.includeErrors
    incExitFcn  = [ { @()logRun(me,'INCORRECT');
@()updateTask(me,tS.INCORRECT) }; exitFcn ];
    breakExitFcn = [ { @()logRun(me,'BREAK_FIX');
@()updateTask(me,tS.BREAKFIX) }; exitFcn ];
else
    incExitFcn  = [ { @()logRun(me,'INCORRECT'); @()resetRun(task) };
exitFcn ];
    breakExitFcn = [ { @()logRun(me,'BREAK_FIX'); @()resetRun(task) };
exitFcn ];
end
```

useTask pattern

When `tS.useTask` is true, check if the task has ended after each trial:

```
if tS.useTask || task.nBlocks > 0
    correctExitFcn = [ correctExitFcn; {@()checkTaskEnded(me)} ];
    incExitFcn     = [ incExitFcn;     {@()checkTaskEnded(me)} ];
    breakExitFcn   = [ breakExitFcn;   {@()checkTaskEnded(me)} ];
end
```

Dynamic next-state pattern

Use `trialVar` or `blockVar` to dynamically choose the next state:

```
% In tS setup:
task.trialVar.values = {'stimulus', 'catch'};
task.trialVar.probability = [0.8 0.2];

% In fixate transition:
@()updateNextState(me, 'trial')
@()testSearchHoldFixation(eT, sM.tempNextState, 'incorrect')
```

skipExitStates

When the task ends (all blocks complete), the state machine loops back from `correct`/`incorrect`/`breakfix` to `prefix`. But we don't want the exit functions (which update the task) to run again. `skipExitStates` prevents this:

```
sM.skipExitStates = {  ...
    'correct',   'prefix'; ...
    'incorrect', 'prefix'; ...
    'breakfix',  'prefix'  ...
};
```

When transitioning from any of these states *to* `prefix`, the exit functions are skipped.

---

see METHODS for more details.

Opticka's User Functions

Adding User Functions

For behavioural tasks, state-machine files specify the experimental states (prefix, correct, incorrect etc.) and functions that run on `enter`/`within`/`transition` and `exit` of *each* state. *Most functions* are specified in the built-in classes like `screenManager`, `taskSequence` etc.

BUT a user can add any extra functions and store information using a customised `userFunctions.m` file. You should edit the standard `userFunctions.m` and copy it to a folder with your protocol. This custom class will be loaded during the task, called `uF`, and you can call functions as your experiment runs, and store variables in properties etc.

You use the Load Functions File... to load this file into your protocol; this will be used when you run your task. You can also use the Edit Functions File... button to open the file in the MATLAB editor.

How to Use these Functions?

Let's add a new function:

```
function drawSomething(me)
  % use screenManager (me.s) to draw some text to the PTB screen
  me.s.drawText('Hello from UserFunctions')
end
```

Remember: `me` refers to itself, in this case `userFunctions`. So `me.s` refers to the `s` property which is automatically set as a handle to `screenManager` the experiment is running under. From the `runExperiment.runTask()`, the userFunctions object is called `uF`.

Lets say we want to run our new `drawSomething()` function during the `fixate` state (on every frame, so we use `withinFcn`). Find the cell array of functions for this state in the `stateInfo.m` file you are using and add our new function to the cell array. Because these cell arrays contain function handles(`@()`), you will need to insert the function name like so: `@()drawSomething(uF)`:

```
%--------------------fix within
fixFcn = {
  @()drawSomething(uF); % our new custom function from our uF object
  @()draw(stims); %draw our stimuli
  @()animate(stims); % animate stimuli for subsequent draw
};
```

This will now run where the `fixFcn` array is, in this case we can see that is `fixate` > `withinFcn`:

```
stateInfoTmp = {
'name'    'next'    'time'  'entryFcn'    'withinFcn'    'transitionFcn'
'exitFcn';
'fixate'  'incorrect' 10    fixEntryFcn   fixFcn      inFixFcn
fixExitFcn;
}
```

Using Variables (Properties)

When using `@()` function handles, you cannot change class variables (properties) directly. Instead you can use "setter" functions that set the properties. Examples of these kinds of functions from the

core classes are `show(stims, 3)`. To see how we do this with our customised userFunctions, lets add a new property to our custom `userFunctions.m` file:

```
properties % ADD YOUR OWN VARIABLES HERE
  myToggle = false
end
```

...and then make a new function to set this property:

```
% Add your functions here!

function doToggle(me,value)
  me.myToggle = value;
end
```

You can now add this function in your state file: `@()doToggle(uF, true)` or `@()doToggle(uF, false)`.

This should allow you to add many custom functions, and store information in variables without needing to edit any of the core opticka classes. If you think you need something more advanced then you can open an issue on github to add functions to the core classes or add a new dedicated class.

IMPORTANT: please don't store important experimental data in the object itself, as although `uF` will be saved into a MAT file, the current userFunctions.m may not be present when loading on a different machine. You can use the structure `tS` or use your own save data function / file.

Available Object Handles

The `userFunctions` class automatically receives handles to all the core experiment objects. These are accessible as properties of `me` (the userFunctions instance):

Property	Object	Description
`me.s`	`screenManager`	PTB screen management
`me.sM`	`stateMachine`	State machine controller
`me.task`	`taskSequence`	Trial randomisation
`me.stims`	`metaStimulus`	Stimulus group
`me.eT`	`eyetrackerCore`	Eye tracker
`me.io`	`ioManager`	Digital I/O
`me.rM`	`rewardManager`	Reward delivery
`me.bR`	`behaviouralRecord`	Performance plot
`me.aM`	`audioManager`	Audio playback
`me.tL`	`timeLogger`	Timing logger

Advanced Patterns

Reading Task Variables

You can read the current trial's task variable values from within userFunctions:

```
function logCurrentTrial(me)
  % get the current trial index
  idx = me.task.totalRuns;
  % read the actual values for this trial
  angle = me.task.outValues{idx, 1};
  % log to the time logger
  me.tL.addMessage([], [], [], sprintf('Angle: %.1f', angle));
end
```

Call this in a state entry function: `@()logCurrentTrial(uF)`.

Modifying Stimuli at Runtime

You can directly access individual stimulus properties via cell indexing:

```
function updateStimulusColour(me, stimIdx, newColour)
  % change the runtime colour of a specific stimulus
  me.stims{stimIdx}.colourOut = newColour;
end
```

Or use the `edit` method on `metaStimulus`:

```
function changeFixationSize(me, newSize)
  edit(me.stims, 2, 'sizeOut', newSize);  % change stimulus 2's size
end
```

Staircase Integration

The base `userFunctions` class includes built-in methods for working with Palamedes staircases:

- `setDelayTimeWithStaircase(me, stim, duration)` — Updates a stimulus's `delayTime` based on staircase output
- `resetDelayTime(me, stim, value)` — Resets the delay time to a fixed value

Example usage in a state file:

```
% In the correct exit function, update the staircase
if ~isempty(task.staircase)
    @()setDelayTimeWithStaircase(uF, 1, stims{1}.delayTime)
end
```

Time Logger Integration

You can log custom events with optional HED (Hierarchical Event Descriptors) tags:

```
function logCustomEvent(me, eventName)
  me.tL.addMessage([], [], [], eventName, '', 'Experimental-note');
end
```

Call in state functions: `@()logCustomEvent(uF, 'stimulus_onset')`.

Subclassing userFunctions

For complex protocols, you can create a subclass of `userFunctions` with a different class name. The DMTS (Delayed Match-to-Sample) protocol demonstrates this pattern with `DMTSFunctions.m`:

```
classdef DMTSFunctions < userFunctions
  properties
    matchIndex = 0
```

```
    sampleIdx = 0
  end

  methods
    function obj = DMTSFunctions(varargin)
      obj = obj@userFunctions(varargin{:});
    end

    function updateStimuliImages(me)
      % custom logic to update stimulus images
      % ...
    end

    function updateLocations(me)
      % custom logic to update stimulus positions
      % ...
    end

    function updateDelayTime(me)
      % custom logic to set delay duration
      % ...
    end
  end
end
```

Then in the state info file, call these methods: `@()updateStimuliImages(uF)`, `@()updateLocations(uF)`, `@()updateDelayTime(uF)`.

Note: When subclassing, the class name must remain `userFunctions` OR you must ensure `runExperiment` loads your custom class. The DMTS protocol achieves this by placing `DMTSFunctions.m` alongside the protocol files.

Opticka's Visual Stimuli

Configuring Stimuli

Opticka uses many different stimulus classes that are collected together using a `metaStimulus` class. The order of the stimulus in the list defines the z-index at which it is drawn (higher number gets drawn on top of lower number). Stimuli collected into a `metaStimulus` can then be controlled as a single object. In this code for example:

```
d = dotsStimulus('XPosition', 5, 'mask', true); %coherent dots
f = fixationCrossStimulus('alpha', 0.5); %fixation cross
stims = metaStimulus();
stims{1} = d;
stims{2} = f;
```

We have two stimuli, and by default `draw(stims)` will draw **both** the coherent dots **and** the fixation cross in one single command. You can then do things like hide/show one or all stimuli:

```
hide(stims); % hide all stims
show(stims, 2); % set the 2nd stimulus (fixationCross) to be shown.
draw(stims); %now only 2nd stimulus will actually be drawn
```

These commands can be run via the stateInfo file. The GUI allows you to add and order stimuli into this metaStimulus manager.

Adding & Editing Stimuli

You can start by selecting a stimulus type from the "Stimulus" menu. You should now see a stimulus panel with individual properties on the right. For example `fixationCross` has a `colour` property which defines the disc colour and `colour2` which defines the cross colour. For each stimulus in the list you can edit all these properties. When you are happy with the settings, you Add it to the stimulus list. The GUI panel on the right updates as you select different stimuli in the stimulus list. You can continue to edit all properties by selecting the stimulus in the list and editing the values.

To move stimuli up or down the list (affecting in what order the stimuli are drawn) use the ↑ and ↓ icons.

Previewing Stimuli

- ▹ Run a single stimulus in an onscreen window to preview how it will look.
- ▹⚡ Benchmark run a single stimulus. See command window for FPS.
- ▸ Run all stimuli in an onscreen window to preview how they will look.
- ▸⚡ Benchmark all stimuli. See command window for FPS.

Common Base Properties

All stimulus classes inherit from `baseStimulus`, which provides a shared set of properties. These can all be used as task variables (`nVar` names):

Property	Default	Description
`xPosition`	0	X position in visual degrees
`yPosition`	0	Y position in visual degrees
`size`	4	Size in visual degrees
`colour`	[1 1 1]	RGB(A) colour, 0–1 range
`alpha`	1	Opacity, 0–1
`angle`	0	Orientation in degrees (0–360)
`speed`	0	Speed in degrees/second
`startPosition`	0	Pre-offset for moving stimuli (degrees)
`delayTime`	0	Delay before display relative to onset (seconds)
`offTime`	Inf	Time to turn off relative to onset (seconds)
`isVisible`	true	Whether to draw

The `*Out` Dynamic Property System

During `setup()`, each public property (e.g. `size`) is cloned into a transient dynamic property (`sizeOut`). The `*Out` version holds the **runtime**, pixel-converted value. When task variables are applied, they write to the `*Out` properties. On `reset()`, these dynamic properties are removed.

This means:

- In the GUI and state info files, you set `size`, `angle`, etc. — these are the *design-time* values
- At runtime, `sizeOut`, `angleOut`, etc. hold the *actual* values being used (which may differ due to task variable changes)
- When using `edit(stims, N, property, value)` during a task, use the `*Out` version: `edit(stims, 3, 'sizeOut', 2)`

The `xyPosition` Magic Variable

When defining task variables, you can use the special name `xyPosition` which allows you to pass both X and Y positions in a single variable value. The value should be a cell array of `[x, y]` pairs:

```
task.nVar(1).name = 'xyPosition';
task.nVar(1).values = {[5 0], [0 5], [-5 0], [0 -5]};
task.nVar(1).stimulus = 1;
```

This sets both `xPositionOut` and `yPositionOut` from a single variable. You can also use the Equidistant Points button to generate these values automatically.

Stimulus Types

Opticka provides the following stimulus classes. Each has unique properties beyond the common base properties listed above. Key distinguishing properties are listed for each type.

Stimulus Class	Type	Description	Key Properties
gratingStimulus	sinusoid / square	Drifting sinusoidal or square-wave grating (procedural shader)	sf, tf, phase, contrast, mask, sigma
gaborStimulus	procedural	Gabor patch with Gaussian envelope (procedural shader)	sf, tf, phase, contrast, spatialConstant
colourGratingStimulus	sinusoid / square	Two-colour grating (procedural shader)	sf, tf, contrast, colour2, baseColour, visibleRate
checkerboardStimulus	checkerboard	Checkerboard pattern (GLSL shader)	sf, tf, contrast, colour2, baseColour
polarGratingStimulus	radial / circular / spiral	Polar (radial/circular/spiral) grating	sf, tf, contrast, colour2, spiralFactor, arcValue, centerMask
polarBoardStimulus	checkerboard	Polar checkerboard (procedural shader)	sf, sf2, tf, contrast, arcValue, centerMask
imageStimulus	picture	Display images (single or directory)	filePath, selection, contrast, crop, circularMask
movieStimulus	movie	Play video files (PTB OpenMovie)	filePath, selection, loopStrategy, mask, circularMask
dotsStimulus	simple	Variable-coherence random dot kinetogram	coherence, density, dotSize, colourType, kill, mask
ndotsStimulus	simple	Alternative limited-lifetime coherence dots	coherence, density, directionWeights, drunkenWalk, interleaving
barStimulus	solid / checkerboard / random	Drifting bar stimulus (for RF mapping)	barWidth, barHeight, contrast, sf, visibleRate
spotStimulus	simple / flash	Simple disc/spot (gluDisk)	contrast, flashColour, flashTime
discStimulus	simple / flash	Procedural smoothed disc with edge smoothing	contrast, sigma, smoothMethod, flashTime
[illegible]	[illegible]	[illegible]	[illegible]

Grating Family Properties

The grating stimuli (`gratingStimulus`, `gaborStimulus`, `colourGratingStimulus`, `checkerboardStimulus`, `polarGratingStimulus`, `polarBoardStimulus`) share these common properties:

Property	Default	Description
`sf`	1	Spatial frequency (cycles/degree)
`tf`	1	Temporal frequency (Hz)
`phase`	0	Grating phase
`contrast`	0.5	Contrast, 0–1
`direction`	0	Object motion direction (degrees)
`reverseDirection` / `driftDirection`	false	Reverse drift direction
`phaseReverseTime`	0	Phase reversal interval (seconds); 0 = no reversal
`phaseOfReverse`	180	Phase offset for reversal
`rotateTexture`	true	Rotate texture vs. patch
`correctPhase`	false	Phase relative to centre

Dots Stimulus Properties

Property	Default	Description
`coherence`	0.5	Motion coherence, 0–1
`density`	100	Dots per degree²
`dotSize`	0.05	Dot width (degrees)
`kill`	0	Limited-lifetime kill fraction
`colourType`	'randomBW'	Dot colouring mode: `simple`, `random`, `randomN`, `randomBW`, `randomNBW`, `binary`

Accessing Individual Stimuli

Use cell indexing on the `metaStimulus` object to access individual stimulus objects directly:

```
stims{1}          % first stimulus object
stims{2}.angleOut % runtime angle of second stimulus
stims{3}.filePath % image path of third stimulus
stims.n           % number of stimuli in the group
```

You can call methods on individual stimuli too:

```
stims{1}.resetTicks()          % reset frame counters
stims{3}.randomiseSelection    % randomise image selection for imageStimulus
```

Opticka's Independant Variable Manager

Configuring Variables

Opticka uses the taskSequence class to specify one or more variable names and values [`nVar`] that can be randomised using balanced blocked repetition and applied to one or multiple stimuli during a task. The taskSequence class can also specify independent trial level [`trialVar`] and block level [`blockVar`] randomisation values. In addition, you can add a staircase (needs Palamedes toolbox), to run alongside the taskSequence (see the `Saccadic Countermanding` core protocol for an example of using a staircase).

nVar Structure

Each independent variable (`nVar`) has the following fields:

Field	Description
`.name`	The stimulus property name (e.g. `'angle'`, `'size'`, `'sf'`)
`.values`	Array of values for this variable. Can be numeric or cell arrays for complex types.
`.stimulus`	[optional] Index (or array of indices) of which stimulus/stimuli to apply this variable to
`.modifier`	[optional] modifier string applied to other stimuli (see Variables)

Value types

Values can be:

- **Numeric arrays**: `[-25 0 25]` — each value is one condition
- **Cell arrays**: `{[1 0 0], [0 1 0]}` — for RGB colours, `{[5 0], [0 5], [-5 0]}` — for `xyPosition` values etc.

`randomiseTask(task)` will generate the randomisation table like so (3 angles × 2 colours per block = **6** conditions; 2 blocks = **12** total trials):

Angle	Colour	Index	IdxAngle	IdxColour	Trial Factor	Block Factor
−25	1 0 0	1	1	1	'none'	'none'
0	1 0 0	3	2	1	'none'	'none'
25	0 1 0	6	3	2	'none'	'none'
25	1 0 0	5	3	1	'none'	'none'
−25	0 1 0	2	1	2	'none'	'none'
0	0 1 0	4	2	2	'none'	'none'
−25	0 1 0	2	1	2	'none'	'none'
0	0 1 0	4	2	2	'none'	'none'
−25	1 0 0	1	1	1	'none'	'none'
25	0 1 0	6	3	2	'none'	'none'
0	1 0 0	3	2	1	'none'	'none'
25	1 0 0	5	3	1	'none'	'none'

Opticka's task function can then use this table to assign variables on each trial to the defined stimuli and generate strobed words to send to external equipment. See core protocols like `OrientationTuning.mat` to get a working example of this in action.

Block and Trial level independent factors

You can set [Block Values] and [Trial Values] in the UI and assign probabilities. These are assigned independently of any variables. So for example setting trial values: `{'a','b'}` trial probabilities: `{0.3 0.7}` would randomly assign either a or b in a 30:70 probability to each trial. Block factors have the same assignation, but applies over the blocks. So for example say we have a variable with two values (-10 or +10 degrees) and 5 repeat blocks. The trials are randomised. Separately we assign `{'a','b'}` to trials and `{'x','y'}` to get an experiment table of 5 blocks that may look like this:

Var	Idx T	rialV Bl	ockV
10 2 b y			
−10 1 b y			
10 2 a x			
−10 1 b x			
10 2 a x			
−10 1 b x			
10 2 a x			
−10 1 b x			
10 2 a y			
−10 1 a y			

Using trial/block factors in state files

Trial and block factors can drive state machine behaviour. For example, to randomly interleave stimulus and catch trials:

```
task.trialVar.values = {'stimulus', 'catch'};
task.trialVar.probability = [0.8 0.2];
```

Then use `@()updateNextState(me, 'trial')` in the transition function to set the next state based on the current trial's factor value.

Trial- and Block-factors can be used to drive state machine functions if you need it (`Saccadic_Doublestep.mat` is an example protocol).

Here is a second example:

```
task.blockVar.values={'A','B'};
task.blockVar.probability = [0.6 0.4];
```

For each block we assign either A or B with a 60:40% probability. Trial-level factors are specified in the same way, but for each trial independently of the blocks.

```
task.trialVar.values={'Y','Z'};
task.trialVar.probability = [0.5 0.5];
```

Angle	Colour	Index	IdxAngle	IdxColour	Trial Factor	Block Factor
−25	1 0 0	1	1	1	Z	A
0	1 0 0	3	2	1	Y	A
25	0 1 0	6	3	2	Y	A
25	1 0 0	5	3	1	Y	A
−25	0 1 0	2	1	2	Z	A
0	0 1 0	4	2	2	Y	A
−25	0 1 0	2	1	2	Y	B
0	0 1 0	4	2	2	Z	B
−25	1 0 0	1	1	1	Z	B
25	0 1 0	6	3	2	Y	B
0	1 0 0	3	2	1	Y	B
25	1 0 0	5	3	1	Y	B

Staircase Procedures

Opticka can run a Palamedes staircase alongside the task sequence. This allows you to adaptively vary a stimulus parameter (e.g. contrast threshold) based on the subject's performance. The staircase is configured in the task sequence and accessed via `task.staircase`.

Example usage in a `userFunctions.m`:

```
function setDelayTimeWithStaircase(me, stim, duration)
  % use the staircase to adjust a delay time
```

```
  me.stims{stim}.delayTime = duration;
end
```

See the `Saccadic Countermanding` core protocol for a full working example of staircase integration.

Variable modifiers

Variables can have modifiers, best explained by example:

1. Name = `angle`
2. Values = `[-90 0 90]`
3. Affects = `1`
4. Modifier = `2; 90`

In this case, angle is varied –90° 0° 90° for stimulus 1. Stimulus 2 has the modifier 90 applied, so for example if stimulus 1 = `-90°` then stimulus 2 = `-90° + 90° = 0°`.

Modifiers can also be string commands:

1. `2; 'shift(2)'` - shift +2 places in the Value list. For example say for stimulus 1 `Values = [-90 -45 0 45 90 135 180]` and for this trial the fourth value `45` is randomly selected. For stimulus 2 we then shift two places to fetch the sixth value `135`. Shift wraps from end to start (e.g. if seventh value `180` was selected for stimulus 1 then second value `-45` would be selected for stimulus 2), and you can use negative values (`-1` would select the third place `0` for example).
2. `2; 'invert'` — take the current value and invert it, so if the current value is `+10°` then invert will make stimulus 2 `-10°`.
3. `2; 'xvar(10, 0.5)'` — For `xyPosition` variables you can add a variable x position. In this case whatever the X position is, `xvar(10)` will add or subtract (50% probability) 10°. So for example if X position is `0°` then the modifier could result in `+10°` or `-10°` with a 50% probability. Change `0.5` to change the probability.
4. `2; 'yvar(10, 0.5)'` — Same as `xvar` but for Y axis.
5. `2; 'xoffset(5)'` — For `xyPosition` add a fixed X position offset, so in this case add 5° to whatever value the X axis position is.
6. `2; 'yoffset(5)'` — For `xyPosition` add a fixed Y position offset, so in this case add 5° for stimulus 2 to whatever value the Y axis position is of stimulus 1.

Modifier summary table

Modifier	Syntax	Description
Numeric offset	`2; 90`	Add a fixed value
Shift	`2; 'shift(N)'`	Move N places in the value list (wraps)
Invert	`2; 'invert'`	Negate the current value
X variable	`2; 'xvar(d, p)'`	Add/subtract d on X axis with probability p
Y variable	`2; 'yvar(d, p)'`	Add/subtract d on Y axis with probability p
X offset	`2; 'xoffset(d)'`	Add fixed d on X axis
Y offset	`2; 'yoffset(d)'`	Add fixed d on Y axis

stimulusTable vs. nVar

There are two ways to randomise stimulus properties:

nVar (task variables)

Used for **independent variables** that define your experimental conditions. These are logged in the task sequence data, contribute to the randomisation table, and each unique combination generates a trial. Use nVar when you need balanced, repeated measures of specific conditions.

stimulusTable (non-task randomisation)

Used for properties you want to randomise **without** creating task conditions. The randomisation is not logged as a task variable, and does not affect the trial count. This is useful for:

- Training protocols where stimulus properties vary but aren't analysed
- Properties that should vary randomly but are not experimental manipulations
- Reducing predictability without adding experimental conditions

Set `stims.stimulusTable` in the state info file, then call `@()randomise(stims)` in the exit function (before `@()update(stims)`).

Example from Chen et al., 2020 Science — the Saccade-to-Phosphene task randomises target size and colour, but these are not task variables:

```
stims.stimulusTable = { ...
    'sizeOut',  {[1], [2], [3], [4]}; ...
    'colourOut', {[1 0 0 1], [0 1 0 1], [0 0 1 1]} ...
};
stims.choice = 'random';
```

controlTable (Arrow-Key Adjustment)

The `controlTable` allows live adjustment of stimulus properties using arrow keys during the experiment. This is useful for fine-tuning parameters during testing or training.

```
stims.controlTable = { ...
    'sizeOut',  1,  0.5; ...    % up/down by 0.5 degrees
```

```
    'sfOut',    2,  0.1; ...    % up/down by 0.1 c/deg
    'contrastOut', 1, 0.1  ...  % up/down by 0.1
};
stims.tableChoice = 1;  % which stimulus to control
```

The first column is the property name, the second is the stimulus index, and the third is the step size. Use the `override` state (available in all state info files) to enter keyboard control mode.

stimulusSets

`stimulusSets` defines pre-configured groups of stimuli that can be shown together using `showSet(stims, N)`:

```
stims.stimulusSets = { ...
    2, ...          % set 1: show only stimulus 2 (fixation)
    [1 2], ...      % set 2: show stimuli 1 & 2
    [1 2 3] ...     % set 3: show all stimuli
};
stims.setChoice = 1;  % default set
```

Then in the state file:

```
@()showSet(stims, 2)  % show stimuli 1 & 2
@()showSet(stims, 3)  % show all stimuli
```

This is particularly useful in protocols like DMTS where different stimulus subsets are shown at different phases of the trial.

Log or Linear interpolation buttons

You can enter 3 values, `start | end | steps`, and press the log or the lin buttons to interpolate a log or linear range, for example `1 2 5` would get converted to `1 1.1892 1.4142 1.6818 2` when pressing the log button.

Equidistant Points button

This small tool allows you to specify the number of points, distance from center and additional rotation to calculate the correct X and Y positions. For example if you specified 8 points with 12° from the center the tool would calculate `{[12 0], [8.49 8.49], [0 12], [-8.49 8.49], [-12 0], [-8.49 -8.49], [0 -12], [8.49 -8.49]}` for the `xyPosition` values.

This is particularly useful for creating evenly-spaced target positions around a circle, commonly used in saccade and attention paradigms. The rotation parameter allows you to offset the starting angle from 0° (rightward).

Randomisation Algorithms

The randomisation algorithm is set via the **Random Algorithm** dropdown in the GUI. Available options include:

Algorithm	Description
`'twister'`	Mersenne Twister (default, recommended)
`'simdTwister'`	SIMD-oriented Fast Mersenne Twister
`'combRecursive'`	Combined Multiple Recursive
`'multCombRecursive'`	Multiplicative Combined Recursive
`'v5normal'`	Legacy MATLAB 5.0 normal generator

The choice of algorithm affects the stream of random numbers used for trial randomisation. For most experiments, the default Mersenne Twister is sufficient.

Appendix

Detailed Install Instructions

Requirements:

- Latest Psychophysics Toolbox (V3.0.18+) — please ensure it is kept up-to-date. Also consider donating to PTB to ensure its future development: https://www.psychtoolbox.net/#future
- MATLAB 2021a+ (we use property validation extensively so newer MATLABs R2025a+ are better for this). While I would love to support Octave, its `classdef` support is currently incomplete…
- Ubuntu V24.04+ strongly recommended, but also runs under Ubuntu V20.04, macOS and Windows 10+
- Java V21 – we use Java classes for communication via ZMQ and for 2D physics animations in PTB, newer MATLAB's do not bundle Java so download seperately.
- For Alyx, we use an S3-like store and use the minio-cli to send data from MATLAB to the object store.
- Eyelink developer kit for Eyelink eyetrackers. «Class = eyelinkManager»
- Titta Toolbox for Tobii Pro eyetrackers. «Class = tobiiManager»
- Palamedes Toolbox to enable staircase behavioural task control.
- LJM for LabJack T4 / T7 digital I/O devices. «Class = labJackT»
- Exodriver for LabJack U3/6 devices. «Class = labJack»
- For Arduino Uno / Seeduino Xiao, there is **no need to install the official Arduino toolbox**, this hardware is supported using PTB's IOPort built-in. «Class = arduinoManager»

Opticka is tested and mostly used on 64bit Ubuntu 24.04 (in the lab) & macOS 26.x (only development) under MATLAB 2026a. The older LabJack U3/U6 interface (`labJack.m`) currently only works under Linux and macOS (Labjack uses a different interface on Linux/macOS vs. Windows). The newer LabJack T4/T7 interface (`labJackT.m`) does work cross-platform. Linux is **by far the best OS** according the PTB developer Mario Kleiner, and receives the majority of development work from him. It is ***strongly advised*** to use it for all real data collection. My experience is that Linux is *much more* robust and performs better than both macOS or Windows, and it is well worth the effort to use Linux for all PTB experimental computers (reserving macOS or Windows systems for development).

Using the Git repository

Using `git` to install is the recommended route; it makes it easy to update:

- Create a parent folder to hold the code, I use `~/Code/` on Ubuntu and macOS and `C:/Code/` on windows.
- `cd` to that parent folder in the terminal and run

```
git clone https://github.com/iandol/opticka.git
```

- In MATLAB, `cd` to the new `~/Code/opticka` folder and run `addOptickaToPath.m`

To keep opticka up-to-date in the terminal: `git pull` — if you want to make local changes, then please create a new local branch to keep the main branch clean so you can pull without issue. If you do have issues pulling you can either (1) reset the repo losing any local changes: `git fetch -v;`

`git reset --hard origin/master; git clean -f -d; git pull` — (2) stash your changes: `git fetch -v; git stash push; git pull`

Using the ZIP file

I recommend using `git` as you can keep the code up-to-date by pulling from Github, but a ZIP install is a bit easier:

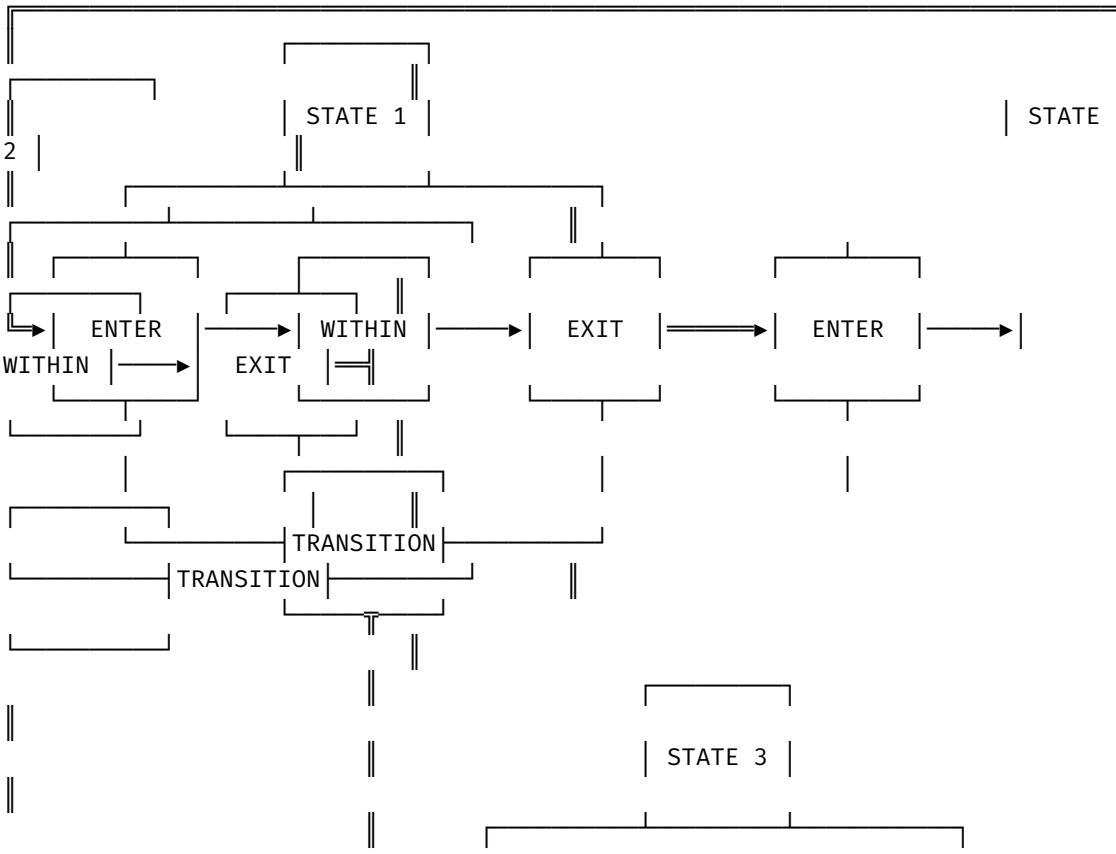
- Download the latest ZIP file: GitHub ZIP File
- Unzip the **contents** of the `opticka-master` folder in the zip to a new folder (I use `~/Code/opticka`). You should end up with something like e.g. `~/Code/opticka/opticka.m` as a path.
- In MATLAB, `cd` to that folder and run `addOptickaToPath.m`.

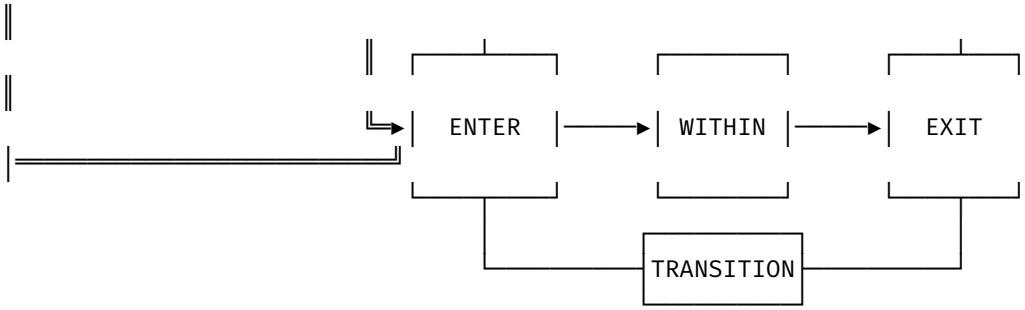
To keep up-to-date you should manually keep downloading and unzipping the newest versions…

See also docs for the stateInfo file.

Useful Task Methods

The state machine (`stateMachine` class) defines states and the connections between them. The state machine can run cell arrays of methods (@( ) anonymous functions) when states are entered (run once), within (repeated on every screen redraw) and exited (run once). In addition there are ways to transition *out* of a state if some condition is met. For example if we are in `[STATE 1]` and the eyetracker tells us the subject has fixated for the correct time, then transition functions can jump us to another state to e.g. show a stimulus.





These various methods control the logic and flow of experiments. This document lists the most important ones used in flexible behavioural task design. It is better for methods to evaluate properties (properties are the variables managed by the class object). Because of this we choose to create methods that alter the properties of each class. For example, `show(stims)` is a method that allows the stimulus manager to show all stimuli in the list; it does this by setting each stimulus' `isVisible` property to `true`. `hide(stims)` hides all stimuli by setting `isVisible` property to `false`, or you could just hide the 3rd stimulus in the list: `hide(stims, 3)`.

For those unfamiliar with object-oriented design, a *CLASS* (e.g. `stateMachine`) is initiated as an *OBJECT* variable (named `sM` during the experiment run, it is an *instance* of the class). **ALL** Opticka classes are **handle classes**; this means if we assign `sM2 = sM` — **both** of these named instances point to the **same** object.

As experiments are run ***inside*** the `runExperiment.runTask()` method, this class refers to *itself* as `me`, so methods that *belong* to `runExperiment` can be called by using `me.myMethod()` or `myMethod(me)` (both forms are equivalent to MATLAB). Other important object instances, for example the `screenManager` class is called via `s`, so to call the method `drawSpot` from our `screenManager` instance `s`, we can use `drawSpot(s)` (or `s.drawSpot()`). You will see below the object names that are available as we run the experiment from `runExperiment`. The `runExperiment` object `me` keeps most of the objects as properties: so `s` is actually also stored in the property `me.screen`, `sM` is stored in the property `me.stateMachine` etc.

Available Object Instances

Object	Class	Description
me	runExperiment	Principal experiment runner
s	screenManager	PTB screen management
sM	stateMachine	State machine controller
task	taskSequence	Trial/block randomisation
stims	metaStimulus	Stimulus group manager
eT	eyetrackerCore subclass	Eye tracker interface
io	ioManager	Digital I/O (strobe/TTL)
rM	rewardManager	Reward delivery
bR	behaviouralRecord	Behavioural performance plot
aM	audioManager	Audio playback
tL	timeLogger	Frame timing and event logging
uF	userFunctions	Custom user functions

Similar named methods?

In some cases `runExperiment` manages an object with similar named methods. For example `runExperiment.updateTask()` will manage the call to `taskSequence.updateTask()`, this is often so that runExperiment can *co-ordinate* among objects and maintain state (when information needs to be shared between objects). If this is not required then we just call the object methods directly, e.g. `drawBackground(s)` uses `screenManager` to run the PTB Screen() functions to draw a background colour (the property `s.backgroundColour`) to the screen. See `DefaultStateInfo.m` and other CoreProtocols state info files for examples of their use.

User Functions Files

If you want to write your own functions to be called by the `stateMachine`, then you can add them to a `userFunctions.m` file, see the docs for details.

Common State Function Patterns

The following table shows the typical function arrays used in each standard state across most CoreProtocols. Use this as a reference when building your own state info files.

prefix — Pre-trial setup

Phase	Typical Functions
Entry	`needFlip(me,true,N)`, `needEyeSample(me,true)`, `hide(stims)`, `resetAll(eT)`, `updateFixationValues(eT,fixX,fixY,[],fixTime)`, `getStimulusPositions(stims)`, `trackerTrialStart(eT,getTaskIndex(me))`, `trackerMessage(eT,['UUID ' UUID(sM)])`
Within	`drawPhotoDiodeSquare(s,[0 0 0])`
Transition	— (time-based, moves to `fixate` after state time)
Exit	`logRun(me,'INITFIX')`, `trackerMessage(eT,'MSG:StartFix')`

fixate — Fixation acquisition

Phase	Typical Functions
Entry	`show(stims,2)` (show fixation stimulus only)
Within	`draw(stims)`, `drawPhotoDiodeSquare(s,[0 0 0])`
Transition	`testSearchHoldFixation(eT,'stimulus','breakfix')`
Exit	`updateFixationValues(eT,[],[],[],stimFixTime)`, `show(stims)`, `trackerMessage(eT,'END_FIX')`

stimulus — Stimulus presentation

Phase	Typical Functions
Entry	`doSyncTime(me)`, `doStrobe(me,true)`
Within	`draw(stims)`, `drawPhotoDiodeSquare(s,[1 1 1])`, `animate(stims)`
Transition	`testHoldFixation(eT,'correct','incorrect')`
Exit	`setStrobeValue(me,255)`, `doStrobe(me,true)`

correct — Correct response / reward

Phase	Typical Functions
Entry	`trackerTrialEnd(eT,tS.CORRECT)`, `needEyeSample(me,false)`, `hide(stims)`, `giveReward(rM)`, `beep(aM,tS.correctSound)`, `logRun(me,'CORRECT')`
Within	`drawPhotoDiodeSquare(s,[0 0 0])`
Exit	`updatePlot(bR,me)`, `updateTask(me,tS.CORRECT)`, `updateVariables(me)`, `update(stims)`, `getStimulusPositions(stims)`, `resetAll(eT)`, `plot(bR,1)`, `checkTaskEnded(me)`

incorrect / breakfix — Error feedback

Phase	Typical Functions
Entry	`trackerTrialEnd(eT,tS.INCORRECT)`, `needEyeSample(me,false)`, `hide(stims)`, `beep(aM,tS.errorSound)`, `logRun(me,'INCORRECT')`
Within	`drawPhotoDiodeSquare(s,[0 0 0])`
Exit	If `tS.includeErrors`: `updatePlot(bR,me)`, `updateTask(me,tS.INCORRECT)`, `updateVariables(me)`, `update(stims)`, `resetAll(eT)`, `plot(bR,1)`, `checkTaskEnded(me)`. Otherwise: `resetRun(task)` instead of `updateTask`.

pause / calibrate / drift — Utility states

Phase	Typical Functions
Entry (pause)	`hide(stims)`, `drawBackground(s)`, `drawTextNow(s,'PAUSED')`, `setOffline(eT)`, `stopRecording(eT,true)`, `needFlip(me,false,0)`, `needEyeSample(me,false)`
Exit (pause)	`startRecording(eT,true)`
Entry (calibrate)	`drawBackground(s)`, `stopRecording(eT)`, `setOffline(eT)`, `trackerSetup(eT)`
Entry (drift)	`drawBackground(s)`, `stopRecording(eT)`, `setOffline(eT)`, `driftCorrection(eT)`

List of Methods

We highlight the main classes and methods that are most useful when building your paradigm:

runExperiment ("me" in the state file)

The principal class object that runs the experiment. It coordinates all other manager objects and maintains the experimental state.

- `enableFlip(me)` || `disableFlip(me)` Enable or disable the PTB screen flip during the update loop.
- `needFlip(me, tf, flipType)` Set whether the screen flips during the update loop. `tf` is `true`/`false`. `flipType` controls the flip mode: `0` = no flip, `1` = auto-flip, `N>1` = flip every Nth frame. Usually called in pause entry (`needFlip(me,false,0)`) and prefix entry (`needFlip(me,true,1)`).
- `needEyeSample(me, value)` On each frame we can check the current eye position (called via `getSample(eT)` of the eye tracker object). This method allows us to turn this ON (`true`) or OFF (`false`).
- `needFlipTracker(me, value)` Enable or disable the eyetracker display window flip synchronisation. Typically set to `false` during correct/incorrect states where we don't need to update the tracker display.
- `var = getTaskIndex(me)` Returns the current trial's variable number by calling `task.outIndex(task.totalRuns)`. A trial variable number is unique to a particular task condition (see the `taskSequence` class which builds these randomised sequences).
- `updateFixationTarget(me, useTask, varargin)` To manage several stimuli together, we use the `metaStimulus` class (object name: `stims`). If you set the `metaStimulus.fixationChoice` parameter you can specify from which stimuli to collect the X and Y positions from. `updateFixationTarget(me)` (calling `getFixationPositions(stims)` internally) iterates through each selected stimulus and returns the X and Y positions assigned to `me.lastXPosition` and `me.lastYPosition`. Having these values we can now assign them using `updateFixationValues(eT)`. Optional arguments: `fixInit`, `fixTime`, `radius`, `strict` — if provided these are passed through to `updateFixationValues`.
- `updateExclusionZones(me, useTask, radius)` Does the same as `updateFixationTarget` but using `metaStimulus.exclusionChoice` to recover the X and Y positions to set up exclusion zones around specified stimuli. `useTask` controls whether to use task-variable-driven positions.
- `updateConditionalFixationTarget(me, stimulus, variable, value, varargin)` Say you have 4 stimuli each with a different angle changed on every trial by the task object, and want the stimulus matching `angle = 90` to be used as the fixation target. This method finds which stimulus is set to a particular variable value and assigns the fixation target X and Y position to that stimulus.
- `updateNextState(me, type)` It is possible to force the stateMachine to jump to a transition to a named state by editing `stateMachine.tempNextState`. This method takes the current taskSequence `trialVar` or `blockVar` and sets the next state name to the value contained for the current trial. So for example you can set `trialVar` to `{'stimulus','catch'}` which randomises each trial with either 'stimulus' or 'catch', then use `@()updateNextState(me,'trial')` to choose this value as the temporary next state name.

- `doSyncTime(me)` Synchronises the experiment timer with the PTB VBL timestamp. Called at the start of the stimulus state to ensure precise timing of stimulus onset.
- `doStrobe(me, tf)` Send or clear a strobe word to the digital I/O hardware. `tf = true` sends the current strobe value, `tf = false` clears it. Typically called at stimulus entry (`doStrobe(me,true)`) and stimulus exit (`doStrobe(me,true)` with a different value).
- `setStrobeValue(me, value)` Sets the strobe code value that will be sent on the next `doStrobe(me,true)` call. For example `setStrobeValue(me,255)` sets the "stimulus off" code. The value sent at stimulus onset is typically the task condition index.
- `logRun(me, message)` Logs a message string into the experiment's run log. Common messages: `'INITFIX'`, `'CORRECT'`, `'INCORRECT'`, `'BREAK_FIX'`. These are stored in the `me.runLog` property.
- `updateVariables(me, excludeList, includeList, useTaskFlag)` Applies the current trial's task variables to the stimuli. Called after `updateTask()` in the exit function to set up the next trial's stimulus parameters. Optional: `excludeList` and `includeList` to filter which variables are applied, `useTaskFlag` to override whether to use the task sequence.
- `updateTask(me, responseCode)` Updates the task sequence with a response code (e.g. `tS.CORRECT`, `tS.INCORRECT`, `tS.BREAKFIX`). This calls `taskSequence.updateTask()` internally, passing the current eyetracker and stimulus info.
- `checkTaskEnded(me)` Checks whether the task sequence has completed all blocks. If so, transitions to the `finished` state. Usually called at the end of correct/incorrect/breakfix exit functions when `tS.useTask` is true.
- `keyOverride(me)` Enables keyboard override mode where pressing arrow keys can manually transition states. Used in the `override` debug state.

---

stateMachine ("sM" in the state file)

The state machine controller that manages the current state, timing, and transitions between states.

- `UUID(sM)` Returns the unique identifier for this state machine run. Useful for logging: `trackerMessage(eT, ['UUID ' UUID(sM)])`.
- `sM.skipExitStates` A cell array of `{fromState, toStatePattern}` pairs. When transitioning from `fromState` to a state matching `toStatePattern`, the exit functions of `fromState` are *skipped*. Commonly set to `{'correct','prefix'; 'incorrect','prefix'; 'breakfix','prefix'}` so that exit functions (which update the task) are not run when the task ends and loops back.
- `sM.tempNextState` A temporary override for the next state name. Set by `updateNextState(me,type)` or manually to dynamically choose the next state. For example, in the SaccadePhosphene protocol, `sM.tempNextState` is used in transition functions: `testSearchHoldFixation(eT, sM.tempNextState, 'incorrect')`.

- `sM.currentUUID` The UUID of the currently executing state. Can be used for tagging: `addTag(stims{1}, sM.currentUUID)`.
- `sM.verbose` Set to `true` to enable verbose logging of state transitions (useful for debugging, typically commented out in production state files).

Task sequence manager ("task" in the state file)

- `updateTask(me, thisResponse, runTime, info)` You can update the task by calling this method. `thisResponse` is the response to the trial (correct, incorrect etc. as you've defined), the runTime is the current time, and info is any other information (often the info given by `runExperiment`). In general, it is better to call `runExperiment.updateTask` which generates the `info` for you using the current information from the eyetracker and stimuli.
- `resetRun(task)` If the subject fails to respond correctly, this method randomises the next trial within the block, minimising the possibility the subject just guesses. If you are at the last trial of a block then this will not do anything.
- `rewindRun(task)` This method rewinds back one trial, allowing you to replay that run again.
- `task.nBlocks` The number of repeat blocks. Can be checked in state files: `if tS.useTask || task.nBlocks > 0`.
- `task.totalRuns` The current trial index (1-based). Useful in `userFunctions` for reading current conditions.
- `task.outValues` A cell array of the actual values for the current trial. Access as `task.outValues{task.totalRuns, variableIndex}`.
- `task.nVar(idx).name` / `task.nVar(idx).values` / `task.nVar(idx).stimulus` / `task.nVar(idx).modifier` Access the independent variable definitions. `.name` is the property name (e.g. `'angle'`), `.values` is the array of values, `.stimulus` is the stimulus index(es) it applies to, `.modifier` is an optional modifier string.
- `task.staircase` Access to the Palamedes staircase object (if a staircase is configured). Used in `userFunctions` methods like `setDelayTimeWithStaircase()`.

The eye tracker ("eT" in the state file)

Opticka provides a unified eye tracker API through `eyetrackerCore` subclasses (`eyelinkManager`, `tobiiManager`, `iRecManager`). All share the same methods. In dummy mode (`eT.isDummy = true`), the mouse position simulates the eye.

Fixation window management

- `updateFixationValues(eT, X, Y, inittime, fixtime, radius, strict)` This method allows us to update the `eT.fixation` structure property; we pass in the various parameters that define the fixation window:
  - `X` = X position in degrees relative to screen centre [0, 0]. +X is to the left.
  - `Y` = Y position in degrees relative to screen centre [0, 0]. +Y is upwards.

- `inittime` = How long the subject is allowed to search before their eye enters the window.
- `fixtime` = How long the subject must keep their eyes within the fixation window.
- `radius` = if this is a single value, this is the circular radius around `X,Y`. If it is 2 values it defines the width × height of a rectangle around `X,Y`.
- `strict` = if strict is `true` then do not allow a subject to leave a fixation window once they enter. If `false` then a subject may enter, leave and re-enter without failure (but must still keep the eye inside the window for the required time).

Note that this command implicitly calls `resetFixation(eT)` as any previous fixation becomes invalid.

· `updateExclusionZones(eT, x, y, radius)` Sets up exclusion zones at the given X,Y positions with the given radius. Exclusion zones are areas where the subject must *not* look. If the eye enters an exclusion zone, the `testSearchHoldFixation` or `testHoldFixation` methods will return the fail state.

· `resetFixation(eT, removeHistory) || resetFixationHistory(eT) || resetFixationTime(eT) || resetExclusionZone(eT) || resetFixInit(eT)`

- `resetFixation` resets all fixation counters that track how long a fixation was held for. Pass `removeHistory` == true also calls `resetFixationHistory(eT)` to remove the temporary log of recent eye positions (this is useful for online plotting of the recent eye position for the last trial etc.)
- `resetFixationTime` only reset the fixation window timers.
- `resetExclusionZone`: resets (removes) the exclusion zones.
- `resetFixInit`: *fixInit* is a timer that stops a saccade *away* from a screen position to occur too quickly (default = 100ms). This method removes this test.

· `resetAll(eT)` Convenience method that calls `resetFixation(eT,true)`, `resetExclusionZone(eT)`, and `resetFixInit(eT)`. The most commonly used reset — called in prefix entry and correct/incorrect exit functions.

· `resetOffset(eT)` Reset the drift offset back to `X = 0; Y = 0` — see `driftOffset(eT)` for the method that sets this value.

Fixation testing (transition functions)

· `testSearchHoldFixation(eT, successState, failState)` Tests whether the subject has acquired and held fixation. Returns `successState` if fixation is held for the required duration, `failState` if the search time expires without fixation, or `''` (empty) if still searching/holding. This is the standard transition function for the `fixate` state.

· `testHoldFixation(eT, successState, failState)` Tests whether the subject maintains fixation (assumes fixation was already acquired). Returns `successState` if the subject is still fixating, `failState` if the eye leaves the fixation window, or `''` if still holding. This is the standard transition function for the `stimulus` state.

Eye sample acquisition

- `getSample(eT)` Gets the current X, Y, pupil data from the eyetracker (or if in dummy mode from the mouse position). This is saved in `eT.x`, `eT.y`, `eT.pupil` and logged to `eT.xAll`, `eT.yAll`, `eT.pupilAll`. NORMALLY you do not need to call this as it is called by `runExperiment` for you on every frame, depending on whether `needEyeSample(me)` was set to `true` or `false`.

Tracker communication

- `trackerMessage(eT, message)` Sends a message string to the eye tracker's recording file. Used for logging trial events: `trackerMessage(eT, 'MSG:Start Fix')`. For Eyelink, this writes to the EDF file.
- `trackerTrialStart(eT, index)` Informs the tracker that a new trial is starting, with the given trial index. For Eyelink, this starts a new recording segment.
- `trackerTrialEnd(eT, code)` Informs the tracker that the trial has ended with the given result code (e.g. `tS.CORRECT`, `tS.INCORRECT`). For Eyelink, this ends the recording segment and logs the result.
- `statusMessage(eT, text)` Sends a status message to the tracker display. Similar to `trackerMessage` but may be displayed on the tracker's operator console.

Tracker display drawing

- `trackerDrawStatus(eT, text, positions, flag, flipFlag)` Draws a status message and stimulus position markers on the eyetracker's operator display. `positions` is typically `stims.stimulusPositions`. `flag` controls whether to draw, `flipFlag` controls whether to flip the tracker display.
- `trackerClearScreen(eT)` Clears the eyetracker's operator display.
- `trackerDrawText(eT, text)` Draws text on the eyetracker's operator display.
- `trackerDrawFixation(eT)` Draws a fixation cross or circle on the eyetracker's operator display at the current fixation position.
- `trackerDrawStimuli(eT, positions)` || `trackerDrawStimuli(eT)` Draws stimulus position markers on the eyetracker's operator display. If `positions` is provided, draws at those locations; otherwise uses `stims.stimulusPositions`.
- `trackerDrawEyePosition(eT)` Draws the current eye position on the eyetracker's operator display. Useful for providing fixation feedback during training.

Recording control

- `startRecording(eT, alsoCalibration)` Starts the eyetracker recording. `alsoCalibration` = `true` also starts the calibration mode recording. Called when exiting the pause state.
- `stopRecording(eT, alsoCalibration)` Stops the eyetracker recording. Called when entering the pause or calibrate state.
- `setOffline(eT)` Puts the eyetracker into offline mode. Called before calibration, drift correction, or when pausing.

- `trackerSetup(eT)` Runs the eyetracker's calibration/setup routine. Called in the calibrate state entry.
- `driftCorrection(eT)` Runs the eyetracker's drift correction procedure. Called in the drift state entry.
- `driftOffset(eT)` Runs the eyetracker's drift offset procedure. Called in the offset state entry. This corrects for small systematic offsets in the eye position reading.

---

metaStimulus ("stims" in the state file)

This class manages groups of stimuli as a single object. Each stimulus can be shown or hidden and metaStimulus can also manage masking stimuli if needed.

- `show(stims, [index])` Show enables a particular stimulus to be drawn to the screen. Without `[index]` all stimuli are shown. You can specify sets of stimuli, e.g. `show(stims, [2 4])` to show the second and fourth stimuli.
- `hide(stims, [index])` The reverse of `show()`. If `index` is empty, hide all stimuli. You can specify sets of stimuli, e.g. `hide(stims, [2 4])` to hide the second and fourth stimuli.
- `showSet(stims, n)` `stims.stimulusSets` is an array of 'sets' of stimuli, for example `{3, [1 3], [1 2 3]}` and calling e.g. `showSet(stims, 3)`, would first hide all the stims (`hide(stims)`), then run `show(stims, [1 2 3])`. This just makes it a bit easier to manage pre-specified stimulus sets.
- `edit(stims, stimulus, variable, value)` Allows you to modify any parameter of a stimulus during the trial, e.g. `edit(stims, 3, 'sizeOut', 2);` sets the size of stimulus 3 to 2°. Note: use the `*Out` version of the property name for runtime changes.
- `draw(stims)` Calls the `draw()` method on each of the managed stimuli. Note if you've used `show()` or `hide()` or `showSet()` then some stimuli may not be displayed. This should be called on every frame update, by setting it to run in the `withinFcn` array.
- `animate(stims)` Calls the `animate()` method for each stimulus, which runs any required per-frame updates (for drifting gratings, moving dots, flashing spots etc.). This should be called after `draw()` for every frame update within a `withinFcn` cell array.
- `update(stims)` For trial based designs, we may change a variable (like size, or spatial frequency), and `update()` ensures the stimulus recalculates for this new variable. This is usually run in a correct/incorrect `exitFcn` block after first calling `updateVariables(me)`. Note some variables (like phase for gratings) do not require an `update()` but are applied immediately; however this depends on their implementation in PTB. `update` can cost a bit of time depending on the stimulus type, so it is not recommended to call it on every frame, but only when necessary.
- `randomise(stims)` Randomises stimulus properties according to `stims.stimulusTable`. This is useful for randomising variables that are *not* task variables, e.g. during training. Call `@()randomise(stims)` before `@()update(stims)` in the exit function.

- `getStimulusPositions(stims, tf)` Retrieves the X and Y positions of all stimuli and stores them in `stims.stimulusPositions`. Typically called in prefix entry and correct exit functions. The optional `tf` argument controls whether to include hidden stimuli.
- `getFixationPositions(stims)` Returns the X and Y positions from stimuli selected by `stims.fixationChoice`. Called internally by `updateFixationTarget(me)`.
- `getExclusionPositions(stims)` Returns the X and Y positions from stimuli selected by `stims.exclusionChoice`. Called internally by `updateExclusionZones(me)`.
- `addTag(stims{N}, tag)` Adds a tag string to the stimulus frame log. Example: `addTag(stims{1}, sM.currentUUID)`.

Accessing individual stimuli

Use cell indexing to access the underlying stimulus objects directly:

```
stims{1}.angleOut    % read the runtime angle of stimulus 1
stims{2}.filePath    % read the image path of stimulus 2
stims{3}.resetTicks  % reset frame counters for stimulus 3
```

Key properties

Property	Description
`stims.choice`	Which stimulus index to use for randomisation (default: random)
`stims.stimulusTable`	Randomisation table for non-task variables
`stims.controlTable`	Arrow-key control table for live variable adjustment
`stims.tableChoice`	Which control table entry to use
`stims.stimulusSets`	Cell array of stimulus index sets for `showSet()`
`stims.setChoice`	Default set index for `showSet()`
`stims.fixationChoice`	Which stimuli return fixation positions
`stims.exclusionChoice`	Which stimuli return exclusion positions
`stims.n`	Number of stimuli in the group

Screen Manager ("s" in the state file)

- `drawBackground(s)` Draws the `s.backgroundColour` to screen.
- `drawPhotoDiodeSquare(s, colour)` Draws a small square in the corner of the screen used by a photodiode to verify stimulus timing. `colour` is RGB, e.g. `[0 0 0]` for off, `[1 1 1]` for on. Typically `[0 0 0]` during fixation/ITI and `[1 1 1]` during stimulus presentation.
- `drawTextNow(s, text)` || `drawText(s, text)` Draws text to the screen. `drawTextNow` draws and flips immediately. `drawText` draws to the buffer (requires a subsequent flip). Used in pause/override states to display status messages.
- `drawGrid(s)` Draws a 1-degree grid overlay on the screen. Activated by the `showgrid` state.

- `drawScreenCenter(s)` Draws a crosshair at the screen centre. Useful for calibration and alignment.
- `flashScreen(s, duration)` Flashes the entire screen white for `duration` seconds. Used in the `flash` state for photodiode calibration.
- `drawTimedSpot(s, size, colour, delay, reset)` Draws a spot that appears after `delay` seconds. Useful for delayed stimulus onset within a single state. `reset` = `true` resets the timer.
- `finishDrawing(s)` Forces a screen flip without the normal frame timing. Used when you need to ensure a draw is visible immediately, e.g. after `drawTimedSpot`.
- `drawMousePosition(s, tf)` || `mousePosition(s, tf)` `drawMousePosition` draws the current mouse position as a cursor on screen. `mousePosition` enables or disables mouse position tracking. Used in the RFLocaliser protocol for mouse-driven experiments.

IO Manager ("io" in the state file)

The `ioManager` class provides a unified interface for digital I/O hardware (strobe words, TTL pulses). In the default configuration it is a dummy that simply logs values. Real hardware subclasses (`plusplusManager`, `labJack`, `dPixxManager`, `arduinoManager`) provide actual implementations.

- `sendTTL(io, value)` Sends a TTL pulse with the given value on the configured output line.
- `sendStrobe(io, value)` Immediately sends a strobe word (multi-bit digital code) with the given value. The value is stored in `io.sendValue`.
- `prepareStrobe(io, value)` Prepares a strobe value for the next trigger event. Does not send immediately.
- `rstart(io)` || `rstop(io)` Start/stop a recording event on the external hardware. For devices like DataPixx that support hardware recording triggers.
- `startFixation(io)` || `correct(io)` || `incorrect(io)` || `breakFixation(io)` Sends named event markers to the external hardware. These are convenience methods that send pre-defined strobe codes for common experimental events.
- `pauseRecording(io)` || `resumeRecording(io)` Pause/resume an ongoing hardware recording.
- `timedTTL(io, pin, duration)` Sends a TTL pulse of `duration` milliseconds on the specified `pin`.
- `io.verbose` Set to `true` to enable verbose logging of all I/O operations.

Behavioural Record ("bR" in the state file)

The `behaviouralRecord` class creates a live performance plot showing success rates, response times, eye positions, and pupil size.

- `updatePlot(bR, me)` Updates the behavioural data from the `runExperiment` object. Reads the state machine for correct/incorrect state names and the eye tracker for fixation timing. Stores response, RT, radius, eye position, pupil data per trial tick.

- `plot(bR, drawNow)` Renders all plot axes with current data: response timeline, running success average, stacked bar of hit/miss/break percentages, RT histograms, eye position scatter, pupil trace. `drawNow` (default `true`) calls `drawnow`. Increments the internal tick counter.
- `bR.correctStateName` The name of the state considered "correct" (default: `'correct'`). Set this in the state info file: `bR.correctStateName = 'correct'`.
- `bR.breakStateName` The name(s) of states considered "break/incorrect" (default: `["breakfix","incorrect"]`). Set this in the state info file: `bR.breakStateName = ["breakfix","incorrect"]`.
- `reset(bR)` Resets all trial data (tick, trials, response, rt, radius, time, inittime, xAll, yAll, comment).

Audio Manager ("aM" in the state file)

The `audioManager` class manages audio playback via PsychPortAudio. It supports WAV file playback and generated beep tones with click-suppression ramping.

- `beep(aM, freq, durationSec, fVolume)` Generates and plays a sine beep tone. `freq` can be a numeric frequency in Hz or one of `'high'` / `'med'` / `'low'`. Defaults: 1000 Hz, 0.15 s, 0.5 volume. Also accepts a vector `[freq, duration, volume]`. Uses cached sound vectors with linear ramp to suppress clicks. Examples:
  - `beep(aM, tS.correctSound)` — play the configured correct sound
  - `beep(aM, tS.errorSound)` — play the configured error sound
  - `beep(aM, [800, 0.2, 0.7])` — 800 Hz for 0.2s at 70% volume
- `play(aM, when)` Plays the loaded sample buffer. Optional `when` for scheduled start time.
- `stop(aM)` Stops audio playback immediately.
- `volume(aM, value)` Sets the playback volume (0–1).

Reward Manager ("rM" in the state file)

The `rewardManager` class provides reward delivery. In the default configuration it is a stub. Actual reward delivery is typically handled via the IO manager or a hardware subclass.

- `giveReward(rM)` Delivers a reward. In practice, this often calls `timedTTL` on the IO hardware to open a solenoid for the configured reward duration.
- `timedTTL(rM, pin, duration)` Sends a TTL pulse of `duration` milliseconds on the specified `pin`. Used for reward solenoid control. Note: in some older protocols, this is called on `lJ` (a LabJack object) instead of `rM`.

Time Logger ("tL" in the state file)

The `timeLogger` class logs frame timing (VBL, show, flip, miss, stimTime) and timestamped event annotations with optional HED tags.

- `addMessage(tL, tick, startTime, exitTime, message, timeType, HED)` Appends a timestamped message to the event log. Arguments:
  - `tick` — frame tick number
  - `startTime` — event onset time
  - `exitTime` — event offset time
  - `message` — description string
  - `timeType` — timestamp source label
  - `HED` — HED annotation tag (default: `"Experimental-note"`)
- `plot(tL)` Plots timing summaries: raw frame times, frame-to-frame deltas, timing offsets, missed frames, and the message log.
- `messageTable(tL)` Exports the message log as a sorted MATLAB table with columns: Onset, Exit, Duration, Tick, StimulusOn, Message, TimeType, HED.

Touch Manager ("tM" in the state file)

The `touchManager` class manages touch screen input, providing a similar API to the eye tracker for touch-based experiments. It wraps PTB's `TouchQueue*` functions and supports dummy mode (mouse simulation).

Touch window management

- `updateWindow(tM, X, Y, radius, doNegation, negationBuffer, strict, init, hold, release)` Updates touch window parameters. All arguments optional; only provided ones are changed. Supports multiple windows via array inputs. Parameters:
  - `X`, `Y` — window centre in degrees
  - `radius` — window radius in degrees
  - `init` — search/initiation time (seconds)
  - `hold` — required hold duration (seconds)
  - `release` — required release duration (`NaN` = no release required)
  - `strict` — if `true`, cannot leave window once entered
  - `doNegation` — if `true`, touch outside the window fails the trial
  - `negationBuffer` — area around window to check for negation touches
- `resetWindow(tM, N)` Resets `N` touch window parameters to defaults. Default N=1.
- `reset(tM, softReset)` Resets all touch state data (positions, events, hold tracking). `softReset=true` preserves `lastPressed` and event flags.

Touch testing (transition functions)

- `[out, held, heldtime, release, releasing, searching, failed, touch, negation] = testHold(tM, yesString, noString)` Returns `yesString` if the touch hold duration is met, `noString` if negation/failed/not held and not searching, empty string otherwise. This is the touch equivalent of `testSearchHoldFixation` / `testHoldFixation`.

- `[out, ...] = testHoldRelease(tM, yesString, noString)` Like `testHold` but requires `wasHeld && release` for the yes condition. The subject must hold and then release within the window.

Touch event processing

- `getEvent(tM)` Core event processing: reads latest event(s) from the touch queue (or mouse in dummy mode), processes NEW/MOVE/RELEASE types, updates position. Returns the last processed event struct.
- `isTouch(tM, getEvt)` Simple check: is a touch active? Optionally calls `getEvent()` first.
- `checkTouchWindows(tM, windows, getEvt)` Gets latest event and checks if touch is within defined window(s).

Communication (brief reference)

Opticka provides two network communication classes for remote control and telemetry:

dataConnection (TCP/UDP)

A PNET-based TCP/UDP socket manager. Key methods: `open()`, `close()`, `read()`, `write()`, `readVar()`, `writeVar()`, `checkData()`, `sendCommand()` (supports `ping`, `echo`, `put`, `eval`, `get`, `close` commands). Supports auto-server mode for remote evaluation.

jzmqConnection (ZeroMQ)

A JeroMQ-based ZeroMQ connection supporting REQ/REP, PUB/SUB, PUSH/PULL socket types with JSON serialisation. Key methods: `open()`, `close()`, `poll()`, `sendCommand()`, `receiveCommand()`, `sendObject()`, `receiveObject()`, `sendViaProxy()`.

FAQ

- **Question:** My subject gave an incorrect answer, but I don't want to keep repeating the same stimulus.
- **Answer:** Use `resetRun(task)` which chooses another run in the same block and swaps it. Note if we are on the last trial of a block, we cannot swap as we want to preserve repeats per block.

- **Question:** I want to randomise some values of the stimuli but not include them as an independent variable.
- **Answer:** The metaStimulus object contains a `stimulusTable` which allows you to make changes to stimuli without them added to the trial structure. This is useful during training, or if you need randomisation tangential to the task. As an example, in Chen et al., 2020 Science their Saccade-to-Phosphene task randomises the size and colour of the target but this is not used as a task variable. In this case set `stimulusTable` and then call `@()randomise(stims);` in the state machine functions (normally just before you call `@()update(stims);`). This will give randomised size and colour without adding any independent variables.

- **Question:** I need the fixation window to move to match a stimulus position that changes on each trial.
- **Answer:** Set `stims.fixationChoice` to the index of the stimulus that serves as the fixation target. Then call `updateFixationTarget(me)` in the prefix entry function. This reads the stimulus position and updates the eyetracker's fixation window automatically.

- **Question:** I want different trial types (e.g. stimulus vs. catch) randomly interleaved.
- **Answer:** Use `task.trialVar` with the state names as values, e.g. `task.trialVar.values = {'stimulus','catch'}`. Then in the transition function for fixate, use `@()updateNextState(me,'trial')` to dynamically set the next state based on the trial variable.

- **Question:** How do I adjust stimulus parameters live during the experiment?
- **Answer:** Use `stims.controlTable` and `stims.tableChoice` to set up arrow-key adjustable variables. Then use the `override` state (available in all state info files) to adjust parameters with arrow keys between trials.

- **Question:** I want to send TTL markers to external recording hardware.
- **Answer:** Use the `io` (ioManager) object. `sendStrobe(io, value)` sends a multi-bit code, `sendTTL(io, value)` sends a single TTL pulse. Call `doStrobe(me, true)` at stimulus onset and `setStrobeValue(me, 255); doStrobe(me, true)` at offset to mark stimulus timing.

- **Question:** How do I run a staircase alongside the task sequence?
- **Answer:** Configure a staircase in the task sequence (requires Palamedes toolbox). Then in your `userFunctions.m`, use `setDelayTimeWithStaircase(me, stim, duration)` to update a stimulus property based on the staircase value. See the Saccadic Countermanding core protocol for an example.

- **Question:** I'm running a touch-screen experiment. How do I replace eye tracker methods?
- **Answer:** Use `touchManager` instead of an eye tracker. The key methods are analogous: `updateWindow(tM, ...)` replaces `updateFixationValues(eT, ...)`, `testHold(tM, ...)` replaces `testSearchHoldFixation(eT, ...)`, and `testHoldRelease(tM, ...)` replaces `testHoldFixation(eT, ...)`.

Definitions

Class

A class is a way to combine a set of related variables (properties) and functions (methods) in a unified object. In MATLAB we use `classdef` to build a class.

Object

A class is a kind of thing, but when we want to use that thing, we *instantiate* it into a 'real' object. So calling `s = screenManager` *instantiates* the `screenManager` class as an object called `s`.

Method

The name given by MATLAB to a function contained in a class.

Handle class

A MATLAB class that inherits from `handle`. All instances of a handle class that point to the same object share the same data. If `a = screenManager` and `b = a`, then changing `a.someProperty` also changes `b.someProperty` — they are the *same* object.

Anonymous function

A function defined inline using `@()`, e.g. `@()draw(stims)`. These are used extensively in state info files because they can be stored in cell arrays and called by the state machine. Note: you cannot directly modify object properties inside anonymous functions — use setter methods instead.

Dynamic property (`*Out`)

At runtime, each stimulus property (e.g. `size`) gets a dynamic copy (`sizeOut`). The `*Out` version holds the pixel-converted, task-variable-modified value. When using `edit(stims, N, property, value)`, use the `*Out` property name.